Enterprise Operations Intelligence Platform

End-to-end Business Intelligence pipeline — from raw CSVs to star-schema data mart with dbt, Python & PostgreSQL

Python | PostgreSQL | dbt | Star Schema | CI Validation

5
Data Entities

10+
dbt Models

100%
FK Validated

PROJECT OVERVIEW

01
Ingest

Raw CSV files (regions, customers, products, stores, orders)

02
Clean & Validate

Python pipeline: normalize IDs, types, referential integrity

03
Load

PostgreSQL raw schema via SQLAlchemy chunked loads

04
Transform

dbt: staging views + dimension & fact mart tables

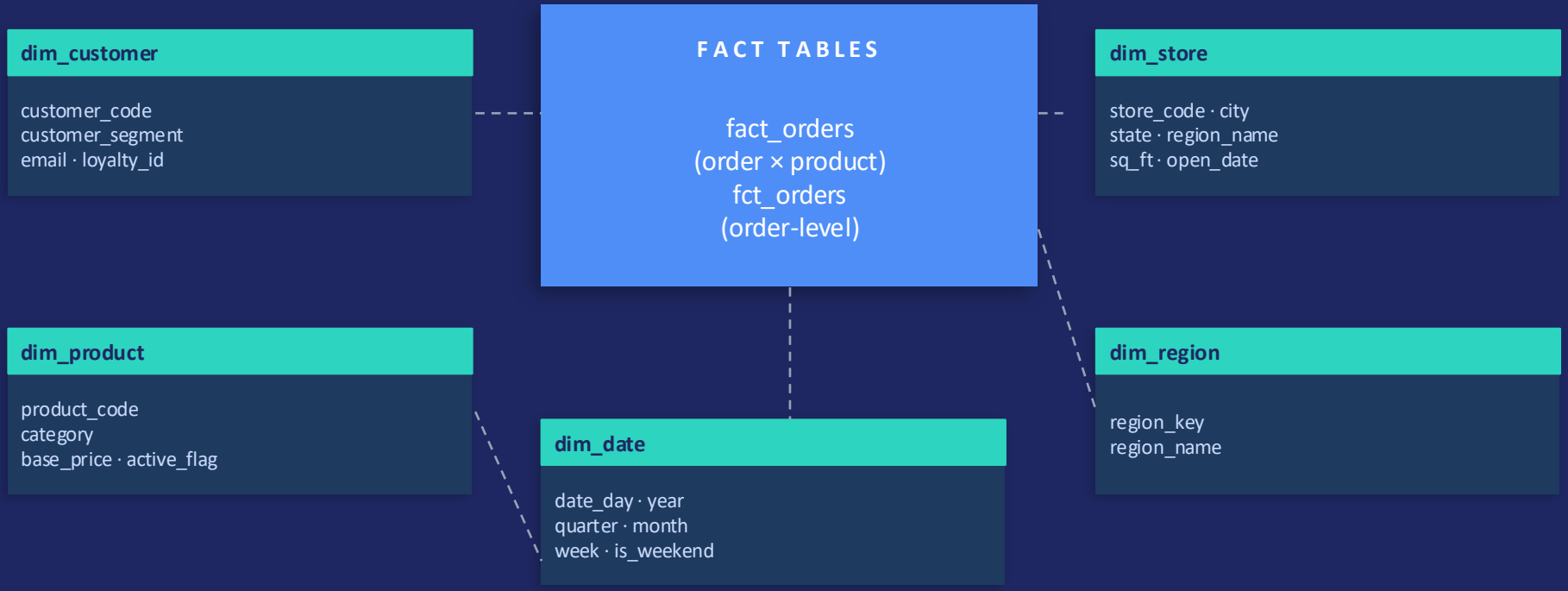
05
Analyze

Star schema ready for dashboards & ad-hoc SQL

Full pipeline executes via a single command: `python python/pipeline.py`

DATA ARCHITECTURE

Star Schema — dbt Mart Layer



PYTHON PIPELINE

ID Standardization

Regex extraction + sentinel IDs (cust-0, store-0, prd-0) for null handling across all entities

Data Type Enforcement

Coercion of dates, ints, floats with coerce-errors; nullable Int64 for safe validation

Duplicate Resolution

Dedup by PK; order line conflicts resolved by reassigning new ord-N codes past current max

Referential Integrity

FK checks at clean-time: invalid store/product codes dropped before DB load, not after

VALIDATION CHECKS

- Schema columns present
- No nulls in required fields
- Correct dtypes enforced
- Allowed values (active_flag)
- Min-value constraints
- Unique key combinations
- Cross-table FK integrity
- Chunked DB load w/ retries

dbt TRANSFORMATION LAYER

STAGING

- stg_customers
- stg_orders_line
- stg_products
- stg_stores
- stg_regions

Views • 1:1 from raw
Type casts + column renames
Null filters applied

DIMENSIONS

- dim_customer
- dim_product
- dim_store
- dim_region
- dim_date

Tables • Deduplicated
Surrogate keys
Date spine generated

FACTS

- fact_orders (order×product)
- fct_orders (order-level)

Tables • Aggregated metrics
Revenue / cost / margin
Line count + total units

dbt tests: not_null · unique · relationships · unique_combination_of_columns | dbt build runs models then tests automatically

KEY TECHNICAL HIGHLIGHTS

Modular Python Architecture

Per-entity clean_*.py modules, orchestrated by pipeline.py. Each function is testable in isolation.

Sentinel ID Design

Null FK values represented as cust-0 / prd-0 / store-0 — preserving data without silent drops.

Chunked DB Loading

SQLAlchemy with pool_pre_ping, retries, and chunked pandas reads for large-file safety.

Two Fact Granularities

fact_orders (order×product) for product mix analysis; fct_orders (order-level) for revenue KPIs.

Dynamic Date Spine

dim_date auto-generates from MIN/MAX order dates in staging — always matches the data range.

Dashboard-Ready SQL

Pre-built dashboard_data/ SQL scripts join all mart tables — plug directly into any BI tool.

TECH STACK & SKILLS DEMONSTRATED

Python 3.9+

- pandas / numpy data cleaning
- SQLAlchemy ORM + connection pooling
- Modular pipeline design
- dotenv config management

PostgreSQL

- Schema management (raw / staging / mart)
- Chunked bulk inserts
- FK constraints & referential integrity
- SQL window functions

dbt Core

- Staging → dimension → fact layering
- ref() / source() lineage
- Schema tests (unique, not_null, FK)
- Materialization config (view / table)

Data Modeling

- Star schema design
- Two fact granularities
- Sentinel ID pattern
- Auto date spine generation

Enterprise Operations Intelligence Platform

What this project demonstrates:

- End-to-end ownership of a production-grade BI pipeline
- Data engineering: cleaning, validation, and relational loading at scale
- Analytics engineering: dbt star schema with full lineage and testing
- Software best practices: modular code, config-driven, error handling

Built with Python · PostgreSQL · dbt · SQLAlchemy · pandas

github.com/AntBap23
linkedin.com/in/anthony-baptiste00
— Let's connect!